

17

Day 15: 闭包 (Closure)

最终交付成果

成果	验证方式
理解作用域链的基本原理	能说出一个函数如何"看到"外部变量
手写一个计数器函数，理解闭包如何"记住"状态	调用多次，计数持续累加而不重置
理解闭包在真实场景中的应用	能举出至少一个闭包解决实际问题的例子
理解闭包可能带来的内存问题	能说出为什么滥用闭包可能导致内存泄漏

最终文件结构

```
text

closure-practice/
└─ closure.js      ← 今天新建，独立练习文件，先不接入作品集项目
```

Step 架构总览

Step	内容	预计用时	目标
Step 1	晨会：回顾 + 目标 + 概念预热	10min	理解闭包要解决什么问题
Step 2	作用域链基础	15min	理解函数如何访问外部变量
Step 3	闭包的形成	15min	手写第一个闭包
Step 4	闭包实战：计数器	15min	用闭包实现"记住状态"的函数
Step 5	闭包的应用场景	15min	认识防抖/节流等真实用例
Step 6	闭包的代价：内存问题	10min	理解潜在风险
Step 7	Code Review + 自评	10min	检验理解

Step 1: 晨会

昨日回顾 (2分钟)

Day 14 你完成了整个第 1 阶段的收尾：最终 Code Review、部署上线到 GitHub Pages、远程分支状态确认，以及阶段总结。

快速自测：

- GitHub Pages 展示的内容来自本地文件还是远程仓库？
- 部署完成后，(`git push`)这个动作多了什么新的意义？

今日目标说明（3分钟）

目标	完成标志
理解作用域链	能说出内层函数为什么能访问外层函数的变量
理解闭包的形成	能写出一个返回函数的函数，并解释"记住"了什么
掌握一个实用闭包场景	用闭包实现一个计数器
了解闭包的风险	能说出为什么闭包可能导致内存无法释放

概念预热（3分钟）

问题： 如果一个函数执行完毕、正常来说它内部的局部变量应该被"清理掉"（这是 Day 5-6 学过的函数作用域的基本认知）。但如果这个函数在结束前，把"访问某个局部变量的能力"交给了另一个函数，并且这个新函数之后还会被调用——你觉得那个局部变量还会被清理掉吗？

PM 视角导入（2分钟）

text

闭包不是一个孤立的语法点，而是 JavaScript 很多"高级用法"的底层原理：

- React 的 `useState`、`useEffect` 内部大量依赖闭包（今天学的内容会在 Week 3-4 学 React 时反复用到，不是学完就丢）
- 防抖（`debounce`）、节流（`throttle`）这类常见的性能优化手段，本质都是闭包的应用
- 面试中"闭包"几乎是前端面试必考题之一

这是一个"当下看起来抽象，但后面会不断用到"的知识点，今天多花点耐心把它吃透，会让后面 React 的学习顺畅很多。

课前预习：

1. 想好两个自测题的答案
2. 想好"局部变量会不会被清理掉"的直觉判断
3. 准备好了，回复"进入 Step 2"